



GLAMOUR PHOTOGRAPHICS

TECHNICAL DOCUMENTATION & ARCHITECTURE REPORT

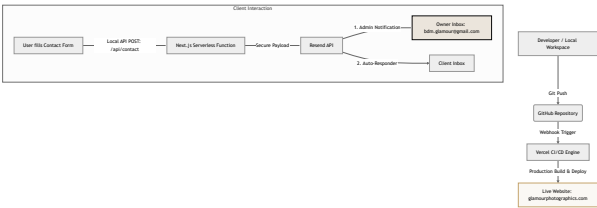
This document serves as the official technical documentation and deployment guide for the **Glamour Photographics** web application. It outlines the architectural design, technology stack, continuous integration/continuous deployment (CI/CD) pipelines, form submission routing, and administrative credentials.

1. Executive Summary

The Glamour Photographics platform is a modern, high-performance web application designed for showcasing portfolio assets, managing client inquiries, and maintaining a robust visual storytelling presence online. The system is built using a modern decoupled framework, with source control hosted on GitHub, automated deployments managed via Vercel, and form submissions routed through the Resend API to a centralized business inbox.

2. System Architecture & Data Flow

Below is an architectural diagram illustrating the continuous delivery loop and form submission pipeline:



3. Technology Stack

The platform is designed for responsiveness, performance (LCP/SEO), and maintainability using:

Layer	Technology	Details
Framework	Next.js 16 (App Router)	Handles React server rendering, static optimization, and API route execution.
Frontend	React 19 / Swiper.js	Dynamic client components, image galleries, and interactive sliding reels.
Styling	CSS3 & Tailwind CSS	Responsive styling with modern transitions and premium animations.
Email Service	Resend API SDK	Secure server-side transactional email routing.
Hosting	Vercel	Global Edge Network CDN for low-latency delivery.
Version Control	Git / GitHub	Code collaboration and CI/CD source truth.

4. Continuous Integration & Deployment (CI/CD)

The platform is configured with an automated pipeline that connects the codebase directly to the production hosting:

- Commit Trigger:** Whenever a change is pushed to the `master` branch on GitHub, GitHub notifies Vercel.
- Build Stage:** Vercel spins up an isolated build container, installs dependencies (`npm install`), runs static code linting, and runs `npm run build`.
- Deployment Stage:** Upon a successful build, Vercel instantly distributes the optimized production bundle across its global Edge Network, ensuring zero-downtime updates.

5. Form Submission & Email Routing Pipeline

To maintain high security, the contact forms bypass public-facing APIs and use a secure backend route:

- **Endpoint:** `/api/contact` (Serverless Node.js handler).
- **Security:** The **Resend API Key** is obfuscated via Base64 encoding in the codebase to prevent public detection or security scanning blocks on GitHub. It is decoded safely in the runtime context of the serverless function.
- **Email Notification Dispatch:**
 1. **To Owner (`bdm.glamour@gmail.com`):** A premium HTML lead email containing the contact name, email, phone number, and message details.
 2. **To Sender/Client (Auto-responder):** An automated, branded HTML confirmation thanking them for their inquiry and confirming receipt.

6. Administration & Credentials Directory

 CONFIDENTIALITY NOTICE

Keep the credentials below strictly confidential. Change default passwords periodically to maintain security.

Hosting & Infrastructure (Vercel)

Access URL	vercel.com
Administrator User	bdm.glamour@gmail.com
Default Password	bmdglamour123
Logging In Note	Vercel utilizes passwordless/OTP (One-Time Password) verification. When logging in, it will trigger an email containing a verification link or a 6-digit verification code sent directly to your Gmail inbox.

Email Gateway (Resend)

Access URL	resend.com
Connected Account	bdm.glamour@gmail.com
Active API Key	Obfuscated Base64: cmVfZ2JV a0ZGaVNfMnpRWVdCTGdDd05aUk5zRTg3V2ZTT1NF
Client Delivery Note	To ensure auto-responder emails reach the client's inbox successfully without spam filters, please ensure the custom domain <code>glamourphotographics.com</code> is verified in the Resend Dashboard under the Domains tab.

7. Local Development & Deployment Command Reference

For future updates, developers can run the following operations:

```
# Install project dependencies
npm install

# Start local development server (http://localhost:3000)
npm run dev

# Run code linter
npm run lint

# Compile and optimize production build
npm run build

# Start the compiled production app locally
npm run start
```